

This document was written to explain how Tuning Maker works. We start with a scale S which is made up of notes N and intervals I . An interval to note a from note b is written $a \leftarrow b$. Naturally, $a \leftarrow b = \frac{1}{b \leftarrow a}$. We can think of the scale as a graph whose notes and intervals are nodes and edges respectively, i.e. $S = (N, I)$. In graph theory an edge can be weighted. Musically, the weight of an interval represents the relative importance of the accuracy of its tuning. For example, it is probably more important that the perfect fifth above a note is in tune than the minor ninth six octave above it, which implies $w(\text{perfect fifth}) > w(\text{minor ninth} + \text{six } 8^{va})$. The weight of an interval is commutative, meaning $w(a \leftarrow b) = w(b \leftarrow a)$, and $w(a \leftarrow a) = 1$. A tuning T of a scale has N notes. The note $r \in N$ is the root note of the scale, whose tuning, $t_r = 1$ by definition. Where $x \neq r$, it's tuning $t_x = f_N(x)$. Let's define this function.

$$f_p(x) = GM\left(\prod_p g(x, p)^{\prod_{p'} w(p \leftarrow p')}\right)$$

P contains the possible notes from which we could travel to note x from. GM stands for the geometric mean. The exponent of each base inside the product is itself the product of all intervals' weights from the notes in $P' (= P)$ to the note p . The sum of exponents whose reciprocal the product is raised to to make this geometric mean is therefore $\sum_p \prod_{p'} w(p \leftarrow p')$. The function g steps through the scale and is defined like so:

$$g(x, p) = \begin{cases} x \leftarrow r, & p = r \text{ or } p = x \\ x \leftarrow p \cdot f_{\{p-x\}}(p), & \text{else} \end{cases}$$

A classic instructive example of the type of problem encountered in musical tuning is the three-note scale of $N = (C, D, A)$. This scale contains the intervals $D \leftarrow C = 9/8$, $A \leftarrow C = 5/3$, and $A \leftarrow D = 3/2$ which are called the major second, major sixth, and perfect fifth respectively. In this example $r = C$, $w(D \leftarrow C) = w(A \leftarrow C) = 1$, and $w(A \leftarrow D) = 2$. Let's work through t_D .

$$\begin{aligned} t_D = f_N(D) &= GM\left(\prod_N S(D, n)^{\prod_{N'} w(n \leftarrow n')}\right) = GM\left(\frac{g(D, C)^{\prod_{N'} w(C \leftarrow n')}}{g(D, D)^{\prod_{N'} w(D \leftarrow n')}}\right) = GM\left(\frac{g(D, C)^1}{g(D, D)^2}\right) = \sqrt[5]{\frac{g(D, C)^1}{g(D, D)^2}} = \sqrt[5]{\frac{D \leftarrow C}{D \leftarrow C^2}} \\ &= \sqrt[5]{\frac{D \leftarrow C}{D \leftarrow A \cdot GM\left(\frac{g(A, C)^1}{g(A, A)^2}\right)^2}} = \sqrt[5]{\frac{D \leftarrow C}{D \leftarrow A \cdot GM\left(\frac{A \leftarrow C}{A \leftarrow C}\right)^2}} = \sqrt[5]{\frac{D \leftarrow C}{D \leftarrow A \cdot C^2}} = \sqrt[5]{\frac{9/8}{(2/3)(5/3)^2}} = \sqrt[5]{\frac{9/8}{9/8^2}} = \sqrt[5]{\frac{9/8}{10/9^2}} = 1.11942 \dots \\ &\cong 195.3\text{¢} \end{aligned}$$

Similarly, $t_A = 1.67497 \dots \cong 893.0\text{¢}$. By comparing the intervals $T(I)$ to I we can see a primary benefit of this way of tuning is that $w(I)$ is inversely proportional to $|T(I) - I|$, which is the relative accuracy of that $T(I)$.

Interval Name	$T(I)$	I	$w(I)$	$ T(I) - I $
Major Second	195.3	203.9	1	8.6
Major Sixth	893.0	884.4	1	8.6
Perfect Fifth	697.7	702.0	2	4.3

While this clear relationship between scale input and resultant tuning breaks down for scales with more notes, thus making the method more black-box-like, it is theoretically capable of tuning any scale. However, notice that the calls to f are recursive – to tune a scale of N notes, there are $(N - 1) \times (N - 1)!$ calls to f . In practice, the computational demands of this many calls make tuning impractical in a human lifetime, meaning recursion must be halted early. There are many ways you might do this, but the way I have chosen is to stop recursion when the total weight of an interval exceeds a lower cutoff c . First, f is changed to take a second argument, which represents the total weight to which an individual interval is raised. Now, $t_n = f_N(x, 1)$ and

$$f_p(x, w) = \prod_p g(x, p, we_p)^{e_p}$$

Since the exponent of each call to g multiplied by the total weight w is now passed as an additional third argument to g , we explicitly write that exponent as $e_p = \frac{\prod_{p'} w(p \leftarrow p')}{\sum_p \prod_{p'} w(p \leftarrow p')}$ for clarity, which is simply an alternative way of writing the geometric mean. Finally,

$$g(x, p, w) = \begin{cases} x \leftarrow r, & p = r \text{ or } p = x \text{ or } w < c \\ x \leftarrow p \cdot f_{\{p-x\}}(p, w), & \text{else} \end{cases}$$

Excluding the new argument w , the only change here is the new third condition, $w < c$, which results in $x \leftarrow r$ and thus halts recursion. This is the way of tuning which Tuning Maker uses.